

Tutorial Week 1

Prerequisites: Python Programming Basics, HTTP Basics

Learning Objectives:

- Understand the basic concepts of LLM and how it works.
- Understand what is an Agent, and what is the difference between LLM and Agent
- Understand the core parameters of the OpenAI API and how to call it.

Part 1: The Basics of Large Language Modeling

1.1 What is LLM

LLM (Large Language Model) is an artificial intelligence model based on deep learning, especially the **Transformer architecture**. LLM (Large Language Model) is an artificial intelligence model based on deep learning, especially the Transformer architecture. It learns the **statistical laws, semantic relationships and contextual logic** of language through training on massive textual data, so as to have the ability to generate and understand natural language.



1.2 What is Prompt

Prompt is a command or context inputted by the user to the LLM, which is used to guide the model to generate the expected output.

Simply put, a **Prompt is a "Question + Context + Requirement" for dialog with the model.**

By carefully designing the Prompt, the performance of the model in a particular task can be greatly improved.

```
messages=[
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Help me write a speech"}
]
```

1.3 Overview of Mainstream Large Models

Current mainstream LLM service providers include:

Provider	Representative models	Features	API Pricing (Input/Output)
----------	-----------------------	----------	----------------------------

Provider	Representative models	Features	API Pricing (Input/Output)
OpenAI	GPT-5	Comprehensive functionality with powerful reasoning	\$2.50/\$10.00 per 1M tokens
Anthropic	Claude 4 Sonnet	Powerful code and Agent capabilities at an unbelievable price!	\$3.00/\$15.00 per 1M tokens
Google	Gemini 2.5 Pro	Excellent multimodal capabilities and price/performance ratio	\$2.50/\$10.00 per 1M tokens

1.4 Token Concept and Counting

Token is the basic unit of text processing in LLM. Understanding tokens is critical for cost control and performance optimization.

<https://huggingface.co/spaces/Xenova/the-tokenizer-playground>

```
# Experiment: Token Counter
import tiktoken

# Initialize OpenAI's tokenizer
encoding = tiktoken.encoding_for_model("gpt-4")

# Test token count for different texts
texts = [
    "Hello, world!",
    "你好, 世界! ",
    "The quick brown fox jumps over the lazy dog.",
    "人工智能正在改变我们的生活方式。"
]

for text in texts:
    tokens = encoding.encode(text)
    print(f"Text: {text}")
    print(f"Number of tokens: {len(tokens)}")
    print(f"Token sequence: {tokens[:10]}...") # Show first 10 tokens
    print("-" * 50)
```

Key Observation:

- English words usually correspond to 1-2 tokens
- Chinese characters usually correspond to 1-2 tokens per word
- Special characters and punctuation also take up tokens.

1.5 Request-Response Model

The LLM API uses the standard HTTP request-response model:

```
# Basic API call workflow
import requests
import json

def call_llm(prompt, api_key):
    """Demonstrate basic structure of LLM API call"""
    headers = {
        "Authorization": f"Bearer {api_key}",
        "Content-Type": "application/json"
    }

    data = {
        "model": "gpt-4o-mini",
        "messages": [
            {"role": "user", "content": prompt}
        ],
        "temperature": 0.7
    }

    response = requests.post(
        "https://api.openai.com/v1/chat/completions",
        headers=headers,
        json=data
    )

    return response.json()
```

1.6 What is an Agent

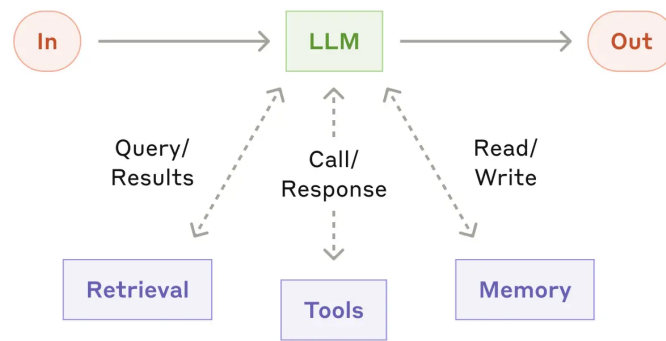
In the field of Artificial Intelligence, **an Agent (Intelligent Body)** usually refers to a system that can **perceive its environment, make decisions and perform actions**.

In the context of the Large Language Model (LLM), **an Agent is a system that allows an LLM to not only "speak" but also "act"**.

What is Agent Architecture



In short, Agent is an augmented LLM.



Part 2: Getting Started

We'll be using OpenAI's official Python library, but we'll be accessing LLM through a third-party proxy service that is compatible with the OpenAI API format. `base_url` is therefore set to an unofficial address, and we'll be able to call non-OpenAI models like Gemini through it as well.

```
# Create virtual environment
python -m venv llm-lab
source llm-lab/bin/activate # Windows: llm-lab\Scripts\activate

# Install necessary packages
pip install openai tiktoken python-dotenv requests
```

```
# .env file
OPENAI_API_KEY=sk-*****
BASE_URL=https://api5.xhub.chat/v1
```

2.1 Environment Preparation

Best practice: don't hardcode any API_KEYS and don't let them appear in any logs (so don't use the `export` or `set` commands to set environment variables, instead use `.env` files to save them)

```
# For windows
$jsonData = @{
    "model"      = "gemini-2.5-flash"
    "messages"   = @(
        @{
            "role"    = "user"
            "content" = "Introduce SUSTech"
        }
    )
}
```

```

    )
    "temperature" = 1.0
    "top_p"       = 1.0
    "stream"      = $true
}

$headers = @{
    "Content-Type" = "application/json"
    "Authorization" = "Bearer $env:OPENAI_API_KEY"
}

Invoke-RestMethod -Uri "$env:BASE_URL/chat/completions" -Method POST -Headers
$headers -Body ($jsonData | ConvertTo-Json)

```

```

# For Linux and MacOS
export $(cat .env | xargs) # Load environment variables
curl ${BASE_URL}/chat/completions \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $OPENAI_API_KEY" \
  -d '{
    "model": "gemini-2.5-flash",
    "messages": [
      {
        "role": "user",
        "content": "Introduce SUSTech"
      }
    ],
    "temperature": 1.0,
    "top_p": 1.0,
    "stream": true
  }'

```

2.2 First LLM application (OpenAI API)

Create `first_llm.py`:

```

from openai import OpenAI
import os
from dotenv import load_dotenv

# Load environment variables
load_dotenv()

# Initialize client
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"), base_url =
os.getenv("BASE_URL"))

def chat_with_llm(user_input):

```

```

"""Single-turn conversation with LLM"""
try:
    response = client.chat.completions.create(
        model="gemini-2.5-flash",
        messages=[
            {"role": "system", "content": "You are a helpful assistant."},
            {"role": "user", "content": user_input}
        ],
        temperature=0.7,
        max_tokens=150
    )
    return response.choices[0].message.content
except Exception as e:
    return f"Error: {str(e)}"

# Test
if __name__ == "__main__":
    print("LLM assistant started! Type 'exit' to end the conversation.")
    while True:
        user_input = input("\nYou: ")
        if user_input.lower() == 'exit':
            break
        response = chat_with_llm(user_input)
        print(f"Assistant: {response}")

```

2.3 OpenAI API core parameters analysis

```

def demonstrate_parameters():
    """Demonstrate the impact of different parameters on output"""

    prompt = "Write a sentence about artificial intelligence"

    # 1. Temperature parameter (creativity)
    for temp in [0.0, 0.5, 1.0]:
        response = client.chat.completions.create(
            model="gemini-2.5-flash",
            messages=[{"role": "user", "content": prompt}],
            temperature=temp,
            n=3 # Generate 3 replies
        )
        print(f"\nTemperature={temp}:")
        for choice in response.choices:
            print(f"  - {choice.message.content}")

    # 2. Max_tokens parameter (length control)
    response = client.chat.completions.create(
        model="gemini-2.5-flash",
        messages=[{"role": "user", "content": "Explain machine learning"}],
        max_tokens=50 # Limit output length
    )

```

```

print(f"\nOutput limited to 50 tokens: {response.choices[0].message.content}")

# 3. System message (role setting)
roles = [
    "You are a rigorous scientist",
    "You are a 5-year-old child",
    "You are Shakespeare"
]

for role in roles:
    response = client.chat.completions.create(
        model="gemini-2.5-flash",
        messages=[
            {"role": "system", "content": role},
            {"role": "user", "content": "What is the sun?"}
        ]
    )
    print(f"\nRole: {role}")
    print(f"Answer: {response.choices[0].message.content[:100]}...")

```

2.4 Streaming

Streaming allows to receive responses in real time, enhancing the user experience:

```

def stream_response(prompt):
    """Demonstrate streaming response"""
    stream = client.chat.completions.create(
        model="gemini-2.5-flash",
        messages=[{"role": "user", "content": prompt}],
        stream=True
    )

    print("Assistant: ", end="")
    collected_messages = []
    for chunk in stream:
        if chunk.choices[0].delta.content:
            content = chunk.choices[0].delta.content
            collected_messages.append(content)
            print(content, end="", flush=True)

    print() # Newline
    return "".join(collected_messages)

# Test streaming output
stream_response("Introduce quantum computing in 100 words")

```

Extended reading

1. **OpenAI official documentation:** platform.openai.com/docs

2. **Anthropic Agent Guide:** Building Effective AI Agents

Thought Question:

- How to design an LLM dialog program that remembers chat messages?
- What does `top_p/top_k` mean in parameters?
- How to design Prompt to let LLM learn how to call tools?